



I Built an Open-Source AI Coding Handbook

[Claude Code](#) • [Codex](#) • [Cursor](#) • [Copilot](#) • [MCP](#) • [AGENTS.md](#)

One vendor-neutral, security-conscious reference for tool selection, repository instructions, workflows, and team adoption.

Why swipe

- Tool fit
- AGENTS.md
- Review workflow
- MCP risk
- Team rollout

For developers and engineering teams

Which AI Coding Tool Fits Your Workflow?

Condensed from the handbook's comparison matrix. The full matrix adds team-fit, verification, and configuration context.

Tool	Best suited for
Claude Code	Deep terminal and repository work
OpenAI Codex	OpenAI-first agentic development
Cursor	IDE-first AI coding
GitHub Copilot	GitHub-centered engineering teams
Open workflows	Provider flexibility and control

Compare by workflow, security, integrations, and team fit ->

Your Coding Agent Needs Repository Instructions

The handbook positions AGENTS.md as the main shared repository instruction file when teams want one durable source of truth across coding agents.

- Architecture
- Build and test commands
- Coding standards
- Restricted files
- Security boundaries
- Definition of done

```
# AGENTS.md

## Build
npm ci
npm test

## Rules
- Follow existing architecture
- Never expose secrets
- Run focused tests before completion
```

Keep it small and durable. The docs warn against turning AGENTS.md into a long architecture book or dumping ground.

Give every coding agent the same repository context.

AI Should Follow Engineering Process - Not Replace It

- 1 Inspect the change
- 2 List findings first
- 3 Separate facts from suspicions
- 4 Validate high-risk findings
- 5 Run focused tests
- 6 Require human approval

Prompt card

Review this diff for correctness, authorization risks, regressions and missing tests.

List findings by severity before summarizing.

The handbook's team standard is findings-first review, with human reviewers owning correctness and merge approval.

AI review is advisory. Humans own approval.

MCP Servers Are Privileged Integrations

The handbook frames MCP as powerful and risky because a connected agent can ask external systems for more context or trigger actions through tools.

Before approval

- Identify the implementation and maintainer
- Review requested permissions
- Understand data access
- Inspect credential handling
- Define revocation steps

Safer starting point

- Read-only access
- Specific repositories
- No unnecessary shell execution
- Human approval for writes
- Logged tool calls

Treat MCP access like production access - not like a harmless plugin.

How Engineering Teams Can Adopt AI Safely

The rollout guide is phased: evaluate one tool for one workflow, run a time-boxed pilot, then expand only if the evidence says it is useful and safe.

Discover

->

Pilot

->

Secure

->

Standardize

Train

->

Measure

->

Govern

- Select approved tools and target one workflow first
- Add repository instructions and review shared prompts
- Approve MCP servers and write access separately
- Train a pilot group, measure quality, then expand with controls

Move from individual experimentation to governed team adoption.

Explore the Open-Source Handbook

- Tool comparisons
- Coding workflows
- Prompts and templates
- MCP security
- Team governance
- Angular and Java guidance



Read it • Star it • Fork it • Contribute

Live handbook

ankitparekh007.github.io/ai-tools-cheatsheets

GitHub repository

github.com/AnkitParekh007/ai-tools-cheatsheets

What should I expand next?